



31장 STL (standard template library)

● C/C++ 강의 (240 시간 8시간*30일)

31. STL (standard template library)

31-1. STL 이란?

31-2. Modern C++란?

31-3. C++11 이상의 발전사

31-3-1 C++11: Modern C++의 시작

31-3-2 C++14: 개선과 간소화

31-3-3 C++17: 편의성과 성능

31-3-4 C++20: 새로운 패러다임

31-3-5 C++23: 지속적인 발전

31-3-6 왜 Modern C++인가?

31-4 자동 타입 추론과 초기화

31-4-1 자동 타입 추론: `auto` 와 `decltype`

31-4-2 유니폼 초기화와 `std::initializer_list`

31-4-3 실습: `auto` 를 활용한 변수 선언 및 초기화 최적화

31-4-4 예상 출력

31-5 람다 표현식

31-5-1 람다 표현식 소개

31-5-2 람다 문법과 캡처 방식

31-5-3 람다와 함수 객체 비교

31-5-4 실습: 람다를 활용한 정렬 및 필터링

31-5-5 결론

31-6 범위 기반 for문과 기타 편의 기능

31-6-1 범위 기반 for문 사용법

31-6-2 nullptr 소개

31-6-3 constexpr 소개

31-6-4 if constexpr 소개

31-6-5 실습: 컨테이너 순회 및 constexpr 활용 예제

31-6-6 결론

31-6-7 참고 자료

31-7 STL 개요 및 주요 컨테이너 실습

31-7-1 STL(Standard Template Library) 개요

31-7-2 순차 컨테이너 소개 및 예제	
31-7-3 연관 컨테이너 소개 및 예제	
31-7-4 실습 과제	
31-7-5 결론 및 다음 주제 예고	
31-8 반복자와 컨테이너 순회 실습	
31-8-1 반복자란?	
31-8-2 반복자의 종류	
31-8-3 반복자 관련 함수	
31-8-4 실습: 반복자를 활용한 컨테이너 순회 및 수정	
31-8-5 실습 과제	
31-8-6 결론	
31-8-7 참고 자료	
31-9 컨테이너 어댑터와 우선순위 큐 실습	
31-9-1 컨테이너 어댑터란?	
31-9-2 주요 어댑터 종류	
31-9-3 사용 예 및 동작 원리	
31-9-4 실습: <code>priority_queue</code> 기반 작업 스케줄러	
31-9-5 실습 과제	
31-9-6 참고 자료	
31-10 STL의 효율적인 활용	
31-10-1 컨테이너 선택 기준	
 상황별 구체적인 선택 예시	
31-10-2 메모리 관리 함수	
31-10-3 실습: 대량 데이터 삽입 및 최적화	
31-10-4 실습 과제	
31-10-5 참고 자료	
<code>unordered_map</code> 을 이용한 단어 빈도수 분석 프로그램 과제 설명	
31-11 STL 알고리즘 개요 및 실습	
31-11-1 STL 알고리즘 개요	
31-11-2 비변경 알고리즘	
31-11-3 변경 알고리즘	
31-11-4 실습: <code>find_if</code> 와 <code>transform</code> 활용	
31-11-5 실습 과제	
31-11-6 참고 자료	
31-12 STL 정렬 알고리즘 및 실습	
31-12-1 정렬 알고리즘 개요	
31-12-2 주요 정렬 알고리즘	
31-12-3 사용자 정의 비교자	
31-12-4 실습: 구조체 벡터 정렬	
31-12-5 실습 과제	
31-12-6 참고 자료	

31-13 함수 객체와 std::function

31-13-1 함수 객체(Function Object, Functor)란?

31-13-2 표준 함수 래퍼 `std::function`

31-13-3 콜백 구현과 활용 예

31-13-4 실습: 이벤트 핸들러 시스템 구현

31-13-5 실습 과제

31-13-6 참고 자료

31-14 고급 알고리즘과 병렬 실행

31-14-1 고급 STL 알고리즘 소개

31-14-2 주요 알고리즘 설명

31-14-3 병렬 알고리즘 (C++17~)

31-14-4 실습: `accumulate` 를 활용한 통계 계산

31-14-5 실습: 병렬 알고리즘 테스트

31-14-6 실습 과제

31-14-7 참고 자료

숙제: `std::sort` 와 사용자 정의 비교자를 활용한 데이터 정렬 프로그램

31. STL (standard template library)

31-1. STL 이란?

STL은 C++의 표준 템플릿 라이브러리(Standard Template Library)로, 프로그래밍에서 자주 사용되는 데이터 구조와 알고리즘을 템플릿으로 제공하는 라이브러리입니다. STL은 컨테이너(vector, list, map 등), 반복자(iterator), 알고리즘(sort, find 등)의 세 가지 주요 구성 요소로 이루어져 있습니다. 이를 통해 개발자는 효율적이고 재사용 가능한 코드를 쉽게 작성할 수 있습니다.

31-2. Modern C++란?

- **Modern C++:** C++11, C++14, C++17, C++20, C++23 표준을 기반으로 한 현대적인 C++ 프로그래밍 방식
- 목표:
 - 코드 가독성 및 유지보수성 향상

- 복잡성 감소
- 성능 최적화와 안전성 강화
- 주요 특징:
 - 자동 타입 추론 (auto, decltype)
 - 스마트 포인터 (std::unique_ptr, std::shared_ptr)
 - 람다 표현식
 - 범위 기반 for 루프
 - 표준 라이브러리 확장 (예: <algorithm>, <thread>)

31-3. C++11 이상의 발전사

31-3-1 C++11: Modern C++의 시작

- 주요 기능:
 - auto와 decltype로 타입 추론
 - 람다 표현식: [capture](parameters) { body }
 - 스마트 포인터: 메모리 관리 단순화
 - 범위 기반 for 루프: for (auto& x : container)
 - nullptr: 널 포인터 상수
 - 스레드 지원: <thread>, <mutex>

31-3-2 C++14: 개선과 간소화

- 주요 기능:
 - auto 반환 타입 추론
 - 람다 캡처 초기화
 - std::make_unique 추가
 - 표준 라이브러리 확장 (예: std::exchange)

31-3-3 C++17: 편의성과 성능

- 주요 기능:

- 구조화된 바인딩: `auto [x, y] = pair;`
- `if`와 `switch` 초기화 구문
- `std::optional`, `std::variant`, `std::any`
- 병렬 알고리즘 (`std::execution`)

31-3-4 C++20: 새로운 패러다임

- 주요 기능:
 - 개념(Concepts): 템플릿 제약 조건
 - 범위(Ranges): `<ranges>` 라이브러리
 - 모듈(Modules): 헤더 파일 대체
 - 코루틴(Coroutines)
 - `std::span`, `std::bit_cast`

31-3-5 C++23: 지속적인 발전

- 주요 기능:
 - `std::flat_map`, `std::flat_set`
 - `std::expected` (에러 처리)
 - 개선된 모듈 지원
 - `std::print`, `std::format` 확장

```
# Code 테스트
g++ -std=c++20 main.cpp -o program
g++ -std=c++23 main.cpp -o program
clang++ -std=c++20 main.cpp -o program
```

31-3-6 왜 Modern C++인가?

- 안전성: 메모리 누수 감소, 타입 안전성 강화
- 가독성: 간결하고 직관적인 코드 작성
- 성능: 컴파일 타임 최적화, 병렬 처리 지원
- 생산성: 복잡한 작업을 단순화하는 표준 라이브러리

참고 자료

- C++ Reference: <https://en.cppreference.com>
- CMake 공식 문서: <https://cmake.org/documentation/>
- g++ 문서: <https://gcc.gnu.org/onlinedocs/>
- clang++ 문서: <https://clang.llvm.org/docs/>

31-4 자동 타입 추론과 초기화

C++의 현대적인 기능들 중에서도 가장 중요한 것은 타입 시스템의 유연성입니다. `auto` 와 `decltype` 을 사용한 자동 타입 추론은 코드를 더 간결하고 안전하게 만들어줍니다. 또한 유니폼 초기화와 `std::initializer_list` 를 통해 일관된 초기화 구문을 제공하며, 이러한 기능들을 실제 프로그래밍에서 활용하면 변수 선언과 초기화를 최적화할 수 있습니다.

31-4-1 자동 타입 추론: `auto` 와 `decltype`

1.1. `auto` 사용법

- **정의:** 컴파일러가 변수의 타입을 초기화 식을 통해 추론
- **장점:**
 - 코드 간결성
 - 복잡한 타입 이름 생략
 - 타입 안정성 유지
- **기본 문법:**

```
auto x = 42;           // int
auto y = 3.14;         // double
auto z = "Hello";      // const char*
auto& ref = x;          // int&
auto ptr = &x;          // int*
```

- **주의점:**

- 초기화 필수: `auto x;` 는 컴파일 오류
- 참조와 포인터 명시 필요: `auto&` , `auto*`
- `const` 와 `volatile` 은 상황에 따라 생략 가능

```
const int a = 10;
auto b = a;      // int (const 생략)
auto& c = a;     // const int&
```

1.2. `decltype` 사용법

- **정의:** 표현식의 타입을 추론하여 선언에 사용
- **용도:**
 - 함수 반환 타입 정의
 - 템플릿 프로그래밍
 - 복잡한 타입 재사용
- **기본 문법:**

```
int x = 42;
double y = 3.14;
decltype(x) a = 10;    // int
decltype(x + y) b = 0.0; // double
```

- **`auto` 와의 차이:**
 - `auto` : 초기화 식에서 타입 추론
 - `decltype` : 표현식 자체의 타입 추론
 - 예:

```
auto x = 42;      // int
decltype(42) y = 42; // int
```

- **Trailing Return Type:**

```
auto add(int a, double b) → decltype(a + b) {
    return a + b;
}
```

31-4-2 유니폼 초기화와 `std::initializer_list`

2.1. 유니폼 초기화 ({})

- **정의:** 중괄호 `{ }` 를 사용한 일관된 초기화 방식 (C++11~)
- **장점:**
 - 타입에 관계없이 동일한 문법
 - 암묵적 변환 방지 (Narrowing Conversion 방지)
 - 명시적 초기화 보장
- **예시:**

```
int x{42};           // 직접 초기화
double y{3.14};      // 직접 초기화
std::vector<int> v{1, 2, 3}; // 리스트 초기화
```

- **Narrowing Conversion 방지:**

```
int i{3.14}; // 오류: double → int 축소 변환
int j = 3.14; // 허용: 암묵적 변환
```

- **클래스 초기화:**

```
struct Point {
    int x;
    int y;
};
Point p{1, 2}; // 멤버 순서대로 초기화
```


2.2. `std::initializer_list`

- **정의:** 종괄호 초기화 리스트를 처리하기 위한 표준 라이브러리 클래스
- **용도:**
 - 컨테이너 초기화
 - 사용자 정의 클래스에서 리스트 초기화
- **기본 문법:**

```
#include <vector>
#include <initializer_list>

std::vector<int> v{1, 2, 3}; // std::initializer_list<int>로 변환

class MyClass {
public:
    MyClass(std::initializer_list<int> list) {
        for (auto x : list) {
            data.push_back(x);
        }
    }
private:
    std::vector<int> data;
};

MyClass obj{10, 20, 30}; // std::initializer_list<int>
```

- **주의점:**
 - `std::initializer_list` 는 임시 객체로, 복사 비용 저렴
 - 동일 타입 요소만 허용
 - 클래스 생성자에서 명시적으로 처리 필요

31-4-3 실습: `auto` 를 활용한 변수 선언 및 초기화 최적화

실습 목표

- `auto` 와 유니폼 초기화를 사용해 간결하고 안전한 코드 작성
- `std::initializer_list` 를 활용한 클래스 설계
- 타입 추론과 초기화 최적화 경험

3.1. 프로젝트 구조

```

auto_init/
├── CMakeLists.txt
├── include/
│   └── stats.hpp
├── src/
│   ├── main.cpp
│   └── stats.cpp
└── build/

```

3.2. 코드 작성

`include/stats.hpp`

```

#pragma once
#include <vector>
#include <initializer_list>

class Stats {
public:
    Stats(std::initializer_list<double> values);
    auto mean() const → double;
    auto size() const → size_t;
private:
    std::vector<double> data;
};

```

`src/stats.cpp`

```
#include "stats.hpp"
```

```
Stats::Stats(std::initializer_list<double> values) : data(values) {}
```

```
auto Stats::mean() const → double {  
    if (data.empty()) return 0.0;  
    auto sum = 0.0;  
    for (const auto& x : data) {  
        sum += x;  
    }  
    return sum / data.size();  
}
```

```
auto Stats::size() const → size_t {  
    return data.size();  
}
```

src/main.cpp

```
#include <iostream>  
#include "stats.hpp"
```

```
int main() {  
    // auto와 유니폼 초기화 사용  
    auto numbers = Stats{1.5, 2.5, 3.5, 4.5, 5.5};  
  
    // auto로 반환 타입 추론  
    auto avg = numbers.mean();  
    auto count = numbers.size();  
  
    std::cout << "Numbers count: " << count << "\n";  
    std::cout << "Mean: " << avg << "\n";  
  
    // 추가 테스트: 빈 객체  
    auto empty = Stats{};  
    std::cout << "Empty mean: " << empty.mean() << "\n";  
}
```

```
return 0;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(AutoInit LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include_directories(include)
add_executable(auto_init src/main.cpp src/stats.cpp)
```

31-4-4 예상 출력

```
Numbers count: 5
Mean: 3.5
Empty mean: 0
```

3.5. 실습 과제

- `Stats` 클래스에 최대값/최소값 계산 함수 추가
 - `auto` 와 범위 기반 for 루프 사용
- `std::vector` 대신 다른 컨테이너 (예: `std::array`)로 변경 후 비교
- 잘못된 초기화 (예: Narrowing Conversion) 테스트 및 오류 확인

결론

- `auto` 와 `decltype` : 코드 간결성과 유연성 제공
- 유니폼 초기화: 안전하고 일관된 초기화 보장
- `std::initializer_list` : 리스트 초기화의 강력한 도구

- 실습을 통해 타입 추론과 초기화 최적화 경험

참고 자료

- C++ Reference: <https://en.cppreference.com/w/cpp/language/auto>
- C++_INITIALIZER List: https://en.cppreference.com/w/cpp/utility/initializer_list
- Modern C++ Guidelines: <https://isocpp.github.io/CppCoreGuidelines/>

31-5 람다 표현식

31-5-1 람다 표현식 소개

- 정의: C++11에서 도입된 익명 함수 객체
- 용도:
 - 짧고 간결한 함수 정의
 - 알고리즘 (예: `std::sort`, `std::for_each`)에 인라인 함수 제공
 - 콜백, 이벤트 처리
- 장점:
 - 코드 가독성 향상
 - 지역 변수 캡처 가능
 - 함수 객체 생성 간소화

31-5-2 람다 문법과 캡처 방식

람다 문법

- 구성:

```
[capture](parameters) mutable → return_type { body }
```

- `[capture]`: 외부 변수 캡처 방식
- `(parameters)`: 함수 인자
- `mutable`: 캡처한 변수 수정 가능 여부 (기본: 상수)

- `> return_type` : 반환 타입 (선택, 생략 시 자동 추론)
- `{ body }` : 함수 본문

- 기본 예시:

```
auto add = [](int a, int b) → int { return a + b; };
int result = add(3, 4); // result = 7
```

캡처 방식

- 캡처란?: 람다 외부의 지역 변수를 람다 내부에서 사용
- 캡처 종류:

- 값 캡처 (`=`):

- 변수 복사본 사용
- 기본적으로 상수 (수정하려면 `mutable` 필요)

```
int x = 10;
auto lam = [=]() { return x; }; // x 복사
x = 20;
std::cout << lam(); // 출력: 10
```

- 참조 캡처 (`&`):

- 변수 참조 사용
- 원본 변수 변경 가능

```
int x = 10;
auto lam = [&]() { x += 10; return x; };
std::cout << lam(); // 출력: 20
std::cout << x;    // 출력: 20
```

- 개별 캡처:

- 특정 변수만 캡처

```
int x = 10, y = 20;
```

```
auto lam = [x, &y]() { y += x; }; // x는 값, y는 참조
```

- **혼합 캡처:**

```
int x = 10, y = 20;  
auto lam = [=, &y]() { y += x; }; // 기본 값 캡처, y는 참조
```

- **주의점:**

- 참조 캡처 시 변수 생명주기 확인 (예: dangling reference 방지)
- 불필요한 캡처 최소화 (성능 최적화)

31-5-3 람다와 함수 객체 비교

함수 객체 (Functor)

- **정의:** `operator()` 를 오버로드한 클래스/구조체
- **예시:**

```
struct Adder {  
    int offset;  
    Adder(int o) : offset(o) {}  
    int operator()(int x) const { return x + offset; }  
};  
Adder add_five(5);  
int result = add_five(10); // result = 15
```

- **장점:**
 - 상태 저장 가능
 - 명시적 타입 정의
 - 복잡한 로직 구현 용이
- **단점:**
 - 코드 길어짐
 - 별도 클래스 정의 필요

람다와 비교

특징	람다 표현식	함수 객체
정의 방식	인라인, 익명	명시적 클래스 정의
코드 길이	짧고 간결	상대적으로 길음
상태 저장	캡처로 가능	멤버 변수로 가능
유연성	간단한 작업에 적합	복잡한 로직에 적합
컴파일러 최적화	인라인 최적화 가능	인라인 가능 (구현 의존)

• 람다의 내부:

- 컴파일러가 람다를 익명 함수 객체로 변환
- 예: `auto lam = [](int x) { return x + 1; };` → 익명 `struct` 생성

• 언제 무엇을?:

- 간단한 작업 → 람다
- 복잡한 상태/재사용 → 함수 객체
- 표준 알고리즘 → 람다 선호

31-5-4 실습: 람다를 활용한 정렬 및 필터링

실습 목표

- 람다를 사용해 `std::sort` 로 사용자 정의 정렬 구현
- 람다로 `std::copy_if` 를 사용한 데이터 필터링
- 캡처와 `mutable` 활용

프로젝트 구조

```
lambda_example/  
├── CMakeLists.txt  
├── include/  
│   └── data_processor.hpp  
├── src/  
│   └── main.cpp
```



```
|   └── data_processor.cpp
└── build/
```

코드 작성

include/data_processor.hpp

```
#pragma once
#include <string>
#include <vector>

class DataProcessor
{
public:
    DataProcessor(std::vector<std::string> data);
    auto sort_by_length() → std::vector<std::string>;
    auto filter_by_prefix(const std::string &prefix) → std::vector<std::string>;
    std::vector<std::string> data_;
};
```

src/data_processor.cpp

```
#include "data_process.hpp"
#include <algorithm>

DataProcessor::DataProcessor(std::vector<std::string> data) : data_(data)
{}

auto DataProcessor::sort_by_length() → std::vector<std::string>
{
    auto result = data_;
    std::sort(result.begin(),
              result.end(),
              [](const std::string &a, const std::string &b)
              {
                  return a.length() < b.length();
              });
};
```

```

        return result;
    }
    auto DataProcessor::filter_by_prefix(const std::string &prefix) → std::vector
    <std::string>
    {
        std::vector<std::string> result;
        std::copy_if(data_.begin(),
                     data_.end(),
                     std::back_inserter(result),
                     [prefix](const std::string &s)
                     {
                         return s.find(prefix) == 0;
                     });
        return result;
    }

```

src/main.cpp

```

#include "data_process.hpp"
#include <iostream>
using namespace std;
int main()
{
    vector<string> words = {"apple", "ant", "banana", "bear", "cat"};
    DataProcessor processor(words);

    auto sorted = processor.sort_by_length();
    cout << "original sequence : " << endl;
    for (const auto &word : processor.data_)
        cout << word << endl;

    cout << "Sorted by length : " << endl;
    for (const auto &word : sorted)
        cout << word << endl;

    cout << "Filtered by prefix : " << endl;
    auto filtered = processor.filter_by_prefix("b");

```

```

for (const auto &word : filtered)
    cout << word << endl;

// capture mutable 사용
int min_length = 4;
auto count_long = [=]() mutable
{
    int count = 0;
    for (const auto &word : words)
    {
        if (word.length() >= min_length)
        {
            count++;
        }
    }
    min_length++;
    cout << "min_length" << min_length << endl;
    return count;
};
cout << "Words with length >= 4" << count_long() << endl;
cout << "After mutable change, min_length: " << min_length << endl;
return 0;
}

```

CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)
project(LambdaExample LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include_directories(include)
add_executable(lambda_example src/main.cpp src/data_processor.cpp)

```

예상 출력

Sorted by length:

ant
cat
apple
bear
banana

Filtered by prefix 'b':

banana
bear

Words with length >= 4: 3

After mutable change, min_length: 4

실습 과제

- `DataProcessor`에 새로운 정렬 기준 추가 (예: 사전순 역순)
- 필터링 조건 변경 (예: 특정 길이 이상 단어)
- 참조 캡처를 사용해 외부 변수 수정하는 람다 작성
- 성능 비교: 람다 vs 함수 객체로 동일 작업 구현

31-5-5 결론

- 람다 표현식은 간결하고 강력한 함수 정의 도구
- 캡처 방식(`=`, `&`)을 통해 유연한 변수 접근 가능
- 함수 객체와 비교해 작업 성격에 따라 선택
- 실습으로 알고리즘과 람다의 실제 활용 경험

참고 자료

- C++ Reference: <https://en.cppreference.com/w/cpp/language/lambda>
- C++ Algorithms: <https://en.cppreference.com/w/cpp/algorithm>
- Modern C++ Guidelines: <https://isocpp.github.io/CppCoreGuidelines/>

31-6 범위 기반 for문과 기타 편의 기능

31-6-1 범위 기반 for문 사용법

- 정의: C++11에서 도입된 컨테이너 순회 방식
- 문법:

```
for (auto& x : container) {  
    // x로 요소 접근  
}
```

- `auto`: 타입 추론
- `&`: 참조로 접근 (수정 가능)
- `const auto&`: 읽기 전용
- `container`: 순회 가능한 객체 (예: `std::vector`, 배열 등)
- 장점:
 - 간결한 코드
 - 인덱스 관리 불필요
 - 반복자(iterator)보다 직관적
- 예시:

```
std::vector<int> vec = {1, 2, 3};  
for (auto& x : vec) {  
    x *= 2; // 수정 가능  
}  
// vec: {2, 4, 6}
```

- 주의점:
 - 컨테이너가 비어 있으면 루프 실행 안 됨
 - 복사(`auto x`)는 성능 저하 가능
 - 범위는 `{}`로 초기화된 임시 객체도 가능:

```
for (const auto& x : {1, 2, 3}) {  
    std::cout << x << " ";  
}
```

```
}
```

- 작동하기 위한 최소 조건

범위 기반 for 문이 작동하려면 컨테이너가 다음을 제공해야 합니다:

1. **begin() 및 end() 함수**: 컨테이너의 시작과 끝을 나타내는 반복자를 반환해야 합니다.
2. **반복자(iterator)**: 반복자는 다음 연산을 지원해야 합니다:
 - 전위 증가 연산자(`++it`)
 - 비교 연산자(`it != end`)
 - 역참조 연산자(`it`)

```
#include <iostream>
#include <cstddef> // std::size_t

template <typename T, std::size_t N>
class MyContainer {
private:
    T data[N];

public:
    MyContainer() {
        for (std::size_t i = 0; i < N; ++i) {
            data[i] = static_cast<T>(i + 1);
        }
    }

    // 반복자 정의
    class Iterator {
private:
        T* ptr;

public:
        using iterator_category = std::forward_iterator_tag;
        using value_type       = T;
        using difference_type   = std::ptrdiff_t;
        using pointer           = T*;
        using reference         = T&;
```

```

explicit Iterator(T* p) : ptr(p) {}

reference operator*() const { return *ptr; }
pointer operator->() { return ptr; }

// 전위 증가 연산자
Iterator& operator++() {
    ++ptr;
    return *this;
}

// 후위 증가 연산자
Iterator operator++(int) {
    Iterator temp = *this;
    ++(*this);
    return temp;
}

friend bool operator==(const Iterator& a, const Iterator& b) {
    return a.ptr == b.ptr;
}

friend bool operator!=(const Iterator& a, const Iterator& b) {
    return a.ptr != b.ptr;
}
};

Iterator begin() { return Iterator(data); }
Iterator end() { return Iterator(data + N); }
};

int main() {
    MyContainer<int, 5> container;

    for (int value : container) {
        std::cout << value << " ";
    }
}

```

```
std::cout << std::endl;

return 0;
}
```

31-6-2 nullptr 소개

- **정의:** C++11에서 도입된 널 포인터 상수
- **목적:**
 - 기존 `NULL`의 모호성 해결 (예: `int`와 포인터 구분)
 - 타입 안전성 강화
- **사용법:**

```
int* ptr = nullptr; // 포인터 초기화
if (ptr == nullptr) {
    std::cout << "Null pointer\n";
}
```

- **장점:**
 - 모든 포인터 타입에 사용 가능
 - 함수 오버로드 명확화:

```
void func(int) { std::cout << "int\n"; }
void func(int*) { std::cout << "pointer\n"; }
func(nullptr); // pointer 호출
```

- **주의점:**
 - `nullptr`은 포인터 타입, 정수로 변환 불가
 - `std::nullptr_t`로 타입 정의:

```
using nullptr_t = decltype(nullptr);
```

31-6-3 constexpr 소개

- **정의:** 컴파일 타임 상수 표현식

- 목적:
 - 컴파일 타임 계산으로 성능 최적화
 - 상수 보장

- 사용법:

- 변수:

```
constexpr int square(int x) { return x * x; }
constexpr int result = square(5); // 컴파일 타임 계산: 25
```

- 함수:

```
constexpr double pi = 3.14159265359;
constexpr double circle_area(double r) { return pi * r * r; }
```

- 장점:

- 런타임 오버헤드 감소
- 배열 크기, 템플릿 인자 등에 사용 가능

- 제약:

- `constexpr` 함수는 컴파일 타임에 계산 가능한 코드만 포함
- C++14부터 더 유연 (예: 루프, 지역 변수 허용)

31-6-4 if constexpr 소개

- 정의: C++17에서 도입된 컴파일 타임 조건문

- 목적:

- 템플릿에서 조건부 코드 제거
- 컴파일 타임 분기 최적화

- 문법:

```
template<typename T>
auto process(T value) {
    if constexpr (std::is_integral_v<T>) {
        return value + 1;
    } else {
```

```

        return value + 0.5;
    }
}

```

- **장점:**

- 컴파일 타임에 불필요한 코드 제거
- 타입별 특화 로직 간단 구현

- **예시:**

```

int x = process(10); // x = 11
double y = process(2.5); // y = 3.0

```

31-6-5 실습: 컨테이너 순회 및 constexpr 활용 예제

- **실습 목표:**

- 범위 기반 for문으로 컨테이너 데이터 처리
- `constexpr` 로 컴파일 타임 상수 계산
- `if constexpr` 로 타입별 동작 구현

- **프로젝트 구조:**

```

range_example/
├── CMakeLists.txt
├── include/
│   └── processor.hpp
├── src/
│   ├── main.cpp
│   └── processor.cpp
└── build/

```

- **코드 작성:**

`include/processor.hpp`

```

#pragma once
#include <string>
#include <vector>

```

```

class Processor
{
public:
    Processor(std::vector<std::string> data);
    auto transform() → std::vector<std::string>;
    template <typename T>
    auto compute(T value) const → T
    {
        if constexpr (std::is_integral_v<T>)
            return value * 2;
        else if constexpr (std::is_floating_point_v<T>)
            return value / 2;
        else
            return value;
    }

private:
    std::vector<std::string> data_;
};

```

src/processor.cpp

```

#include "processor.hpp"
#include <algorithm>

Processor::Processor(std::vector<std::string> data) : data_(data) {}
auto Processor::transform() → std::vector<std::string>
{
    auto result = data_;
    for (auto &s : result)
    {
        if (s.empty())
            s = "empty";
        else
            s = s + "_processed"; // string class + "..."
    }
}

```

```

    return result;
}

```

src/main.cpp

```

#include "processor.hpp"
#include <iostream>
using namespace std;

constexpr int factorial(int n)
{
    return n <= 1 ? 1 : n * factorial(n - 1);
}

int main()
{
    // constexpr 테스트
    constexpr int fact = factorial(5); // 컴파일 타임 계산!
    cout << "Factorial(5)" << fact << endl;

    // 범위 기반 for 문
    vector<string> input = {"hello", "", "world"};
    Processor proc(input);

    auto transformed = proc.transform();
    cout << "transformed 데이터: " << endl;
    for (const auto &s : transformed)
        cout << s << endl;

    // if constexpr 테스트
    cout << "Compute tests: " << endl;
    cout << "Int 10 : " << proc.compute(10) << endl;
    cout << "Float 10.0 : " << proc.compute(10.0) << endl;
    cout << "String 'test' : " << proc.compute("test") << endl;

    // nullptr 테스트
    string *ptr = nullptr; // 초기화
    cout << "Pointer is : " << (ptr == nullptr ? "null" : "non-null") << endl;
}

```

```

string a = "Non nullptr!!!";
ptr = &a;
cout << "Pointer is : " << (ptr == nullptr ? "null" : "non-null") << endl;
return 0;
}

```

CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)
project(RangeExample LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include_directories(include)
add_executable(range_example src/main.cpp src/processor.cpp)

```

• 예상 출력:

```

Factorial(5): 120
Transformed data:
hello_processed
empty
world_processed

Compute tests:
Integer 10: 20
Float 10.0: 5
String 'test': test
Pointer is null

```

• 실습 과제:

- `Processor::transform` 에 새로운 변환 로직 추가 (예: 대문자 변환)
- `factorial` 을 `constexpr` 재귀 대신 반복문으로 구현
- `if constexpr` 로 다른 타입 처리 추가 (예: `std::string` 길이 반환)
- `nullptr` 을 함수 인자로 전달해 동작 확인

31-6-6 결론

- 범위 기반 for문은 컨테이너 순회를 간결하게 처리
- `nullptr` 은 안전한 포인터 초기화 제공
- `constexpr` 과 `if constexpr` 는 컴파일 타임 최적화와 유연성 강화
- 실습으로 Modern C++의 편의 기능 활용 경험

31-6-7 참고 자료

- C++ Reference: <https://en.cppreference.com/w/cpp/language/range-for>
- `nullptr`: <https://en.cppreference.com/w/cpp/language/nullptr>
- `constexpr`: <https://en.cppreference.com/w/cpp/language/constexpr>
- Modern C++ Guidelines: <https://isocpp.github.io/CppCoreGuidelines/>

31-7 STL 개요 및 주요 컨테이너 실습

31-7-1 STL(Standard Template Library) 개요

STL은 크게 네 가지 주요 구성 요소를 가지고 있습니다. 먼저 컨테이너는 데이터를 저장하고 관리하는 자료구조입니다. 알고리즘은 정렬이나 탐색과 같은 데이터 처리 기능을 제공합니다. 반복자는 포인터와 유사하게 동작하며 컨테이너의 요소들을 순회하는 데 사용됩니다. 마지막으로 함수 객체는 함수처럼 동작하는 객체로, 알고리즘에서 활용됩니다.

STL은 현대 C++ 프로그래밍에서 강력한 장점들을 제공합니다. 가장 주목할 만한 특징은 메모리 관리의 자동화입니다. 개발자가 직접 메모리 할당과 해제를 관리할 필요 없이, STL 컨테이너가 이를 자동으로 처리합니다. 또한 템플릿을 기반으로 하여 어떤 데이터 타입이든 저장하고 관리할 수 있는 유연성을 제공합니다.

컨테이너는 요소가 추가되거나 제거될 때 자동으로 크기를 조정하며, 저장된 객체의 개수를 실시간으로 추적합니다. 이러한 자동화된 관리 기능은 개발자가 저수준의 메모리 관리 대신 비즈니스 로직에 집중할 수 있게 해줍니다. 더불어 범위 검사와 타입 안전성을 제공하여 프로그래밍 오류를 최소화하고, 표준화된 인터페이스를 통해 코드의 재사용성과 이식성을 크게 향상시킵니다.

31-7-2 순차 컨테이너 소개 및 예제

vector

- 동적 배열, 빠른 접근

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};
    v.push_back(4);
    for (auto& x : v) std::cout << x << " ";
    return 0;
}
```

array

- 고정 크기 배열

```
#include <iostream>
#include <array>

int main() {
    std::array<int, 3> arr = {10, 20, 30};
    for (const auto& x : arr) std::cout << x << " ";
    return 0;
}
```

deque

- 양쪽 삽입/삭제 $O(1)$

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> dq = {2, 3};
    dq.push_front(1);
    dq.push_back(4);
}
```

```

    for (int x : dq) std::cout << x << " ";
    return 0;
}

```

list

- 이중 연결 리스트, 중간 삽입에 적합

```

#include <iostream>
#include <list>

int main() {
    std::list<int> lst = {1, 2, 4};
    auto it = ++lst.begin();
    lst.insert(it, 3); // 1, 2, 3, 4
    for (int x : lst) std::cout << x << " ";
    return 0;
}

```

31-7-3 연관 컨테이너 소개 및 예제

map

- key-value 쌍, 자동 정렬

```

#include <iostream>
#include <map>

int main() {
    std::map<std::string, int> m = {"Alice", 90}, {"Bob", 85};
    for (auto& [name, score] : m) {
        std::cout << name << ": " << score << "\n";
    }
    return 0;
}

```


set

- 중복 없는 집합, 자동 정렬

```
#include <iostream>
#include <set>

int main() {
    std::set<int> s = {3, 1, 2, 2};
    for (int x : s) std::cout << x << " ";
    return 0;
}
```

unordered_map

- 해시 기반 map, 빠른 검색

```
#include <iostream>
#include <unordered_map>

int main() {
    std::unordered_map<std::string, int> um = {"apple", 100}, {"banana", 150};
    for (auto& [key, val] : um) {
        std::cout << key << ": " << val << "\n";
    }
    return 0;
}
```

unordered_set

- 해시 기반 set, 중복 없는 값 저장

```
#include <iostream>
#include <unordered_set>

int main() {
    std::unordered_set<int> us = {5, 3, 1, 3};
}
```

```
for (int x : us) std::cout << x << " ";  
return 0;  
}
```

31-7-4 실습 과제

- `deque` 로 큐 구현
- `unordered_map` 을 이용한 전화번호부
- `set` 을 활용한 중복 제거 예제

31-7-5 결론 및 다음 주제 예고

- STL은 다양한 상황에 맞는 자료구조를 효율적으로 제공

아래는 요청하신 "31-8 반복자와 컨테이너 순회 실습"에 대한 강의 자료입니다. 각 반복자 유형과 `begin()`, `end()`, `rbegin()`, `rend()` 의 사용법, 그리고 반복자를 활용한 컨테이너 순회 및 수정 실습을 포함하고 있습니다.

31-8 반복자와 컨테이너 순회 실습

31-8-1 반복자란?

- **정의:** 반복자(iterator)는 컨테이너의 요소를 순차적으로 접근할 수 있는 객체입니다.
- **역할:** 포인터처럼 동작하여 컨테이너의 요소를 순회하거나 수정할 수 있습니다.
- **장점:**
 - 컨테이너의 내부 구조를 몰라도 요소에 접근 가능
 - 범용 알고리즘과의 호환성
 - 코드의 재사용성과 가독성 향상

31-8-2 반복자의 종류

반복자는 기능에 따라 다음과 같이 분류됩니다:

1. 입력 반복자 (Input Iterator)

- **특징:** 한 방향으로만 이동하며 읽기 전용
- **사용 예:** 입력 스트림 순회

2. 출력 반복자 (Output Iterator)

- **특징:** 한 방향으로만 이동하며 쓰기 전용
- **사용 예:** 출력 스트림에 데이터 쓰기

3. 전방 반복자 (Forward Iterator)

- **특징:** 한 방향으로 이동하며 읽기 및 쓰기 가능
- **사용 예:** 단일 연결 리스트 순회

4. 양방향 반복자 (Bidirectional Iterator)

- **특징:** 앞뒤로 이동 가능하며 읽기 및 쓰기 가능
- **사용 예:** 이중 연결 리스트, `std::map`, `std::set` 등

5. 임의 접근 반복자 (Random Access Iterator)

- **특징:** 임의의 위치로 이동 가능하며 배열처럼 인덱스로 접근 가능
- **사용 예:** `std::vector`, `std::deque`, 배열 등

31-8-3 반복자 관련 함수

표준 컨테이너는 다음과 같은 반복자 관련 함수를 제공합니다:

- `begin()` : 컨테이너의 시작을 가리키는 반복자 반환
- `end()` : 컨테이너의 끝(마지막 요소의 다음)을 가리키는 반복자 반환
- `rbegin()` : 컨테이너의 마지막 요소를 가리키는 역방향 반복자 반환
- `rend()` : 컨테이너의 첫 번째 요소의 이전을 가리키는 역방향 반복자 반환

이를 활용하여 컨테이너를 정방향 또는 역방향으로 순회할 수 있습니다.

31-8-4 실습: 반복자를 활용한 컨테이너 순회 및 수정

예제 1: `std::vector` 정방향 순회 및 수정

```
#include <iostream>
#include <vector>
```

```

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};

    // 정방향 순회하며 각 요소를 2배로 수정
    for (auto it = numbers.begin(); it != numbers.end(); ++it) {
        *it *= 2;
    }

    // 결과 출력
    for (auto it = numbers.begin(); it != numbers.end(); ++it) {
        std::cout << *it << " ";
    }
    return 0;
}

```

예제 2: `std::list` 역방향 순회

```

#include <iostream>
#include <list>

int main() {
    std::list<std::string> fruits = {"apple", "banana", "cherry"};

    // 역방향 순회하며 요소 출력
    for (auto rit = fruits.rbegin(); rit != fruits.rend(); ++rit) {
        std::cout << *rit << " ";
    }
    return 0;
}

```

예제 3: `std::map` 을 이용한 키-값 출력

```

#include <iostream>
#include <map>

int main() {
    std::map<std::string, int> scores = {"Alice", 90}, {"Bob", 85}, {"Charlie", 95}};
}

```

```
// 정방향 순회하며 키-값 출력
for (auto it = scores.begin(); it != scores.end(); ++it) {
    std::cout << it->first << ": " << it->second << std::endl;
}
return 0;
}
```

31-8-5 실습 과제

1. `std::deque<int>` 를 생성하고, 정방향 및 역방향으로 순회하여 요소를 출력해보세요.
2. `std::set<std::string>` 에 여러 문자열을 삽입한 후, 반복자를 사용하여 사전순으로 출력해보세요.
3. `std::vector<int>` 에서 짝수만 출력하는 코드를 작성해보세요.

31-8-6 결론

- 반복자는 다양한 컨테이너를 순회하고 조작할 수 있는 강력한 도구입니다.
- 반복자의 종류와 특성을 이해하면, 적절한 컨테이너와 알고리즘을 선택하는 데 도움이 됩니다.
- `begin()`, `end()`, `rbegin()`, `rend()` 등의 함수를 활용하여 컨테이너를 효율적으로 순회할 수 있습니다.

31-8-7 참고 자료

- [cppreference - Iterator](#)
- [cppreference - Container functions](#)
- [cplusplus.com - STL Iterators](#)

31-9 컨테이너 어댑터와 우선순위 큐 실습

31-9-1 컨테이너 어댑터란?

- **정의:** 기존 컨테이너(`vector`, `deque`, `list` 등)를 기반으로 제한된 인터페이스만 제공하는 래퍼
 - **목적:** 특정 용도에 맞는 추상화된 컨테이너 제공
-

31-9-2 주요 어댑터 종류

1. `stack`

- 후입선출 (LIFO) 구조
- 기본 컨테이너: `deque`
- 주요 함수: `push()`, `pop()`, `top()`

1. `queue`

- 선입선출 (FIFO) 구조
- 기본 컨테이너: `deque`
- 주요 함수: `push()`, `pop()`, `front()`, `back()`

1. `priority_queue`

- 우선순위가 높은 요소가 먼저 나오는 큐
 - 기본 컨테이너: `vector` + `make_heap`
 - 기본 정렬 기준: 최대 힙 (내림차순)
-

31-9-3 사용 예 및 동작 원리

- `stack`: 웹 브라우저의 뒤로가기, 함수 호출 스택
 - `queue`: 작업 대기열, 프린터 출력 순서
 - `priority_queue`: 작업 우선순위, 경로 탐색 알고리즘 (예: Dijkstra)
-

31-9-4 실습: `priority_queue` 기반 작업 스케줄러

```
#include <iostream>
#include <queue>
#include <string>
#include <vector>
using namespace std;
```

```

class Task
{
public:
    string name;
    int priority;

    // rhs( right hand side) - lhs(left hand side ), other
    bool operator<(const Task &rhs) const
    {
        return priority > rhs.priority; // 오름차순
    }
};

int main()
{
    priority_queue<Task> scheduler;

    scheduler.push({"Write report", 2});
    scheduler.push({"Fix bug", 3});
    scheduler.push({"Email team", 1});

    cout << "작업 처리 순서" << endl;
    while (!scheduler.empty())
    {
        Task t = scheduler.top();
        scheduler.pop();
        cout << t.name << " 우선순위 " << t.priority << endl;
    }
    return 0;
}

```

31-9-5 실습 과제

1. `stack` 을 사용해 괄호 짝 검사 프로그램 작성
2. `queue` 로 동물 병원 접수 시스템 구현
3. `priority_queue` 에 사용자 정의 비교 함수 추가하여 오름차순 스케줄링 구현

31-9-6 참고 자료

- [cppreference - Container adapters](#)
- [cplusplus.com - priority_queue](#)

31-10 STL의 효율적인 활용

31-10-1 컨테이너 선택 기준

STL에서 어떤 컨테이너를 사용할지는 자료의 삽입/삭제 위치, 접근 방식, 메모리 사용량, 정렬 여부에 따라 달라집니다.

상황별 구체적인 선택 예시

상황	추천 컨테이너	이유
많은 양의 데이터를 빠르게 추가하고 인덱스로 접근해야 함	<code>vector</code>	메모리 연속성, 랜덤 접근 $O(1)$, 캐시 효율성 높음
중간 위치에 자주 삽입/삭제가 발생함	<code>list</code>	중간 삽입/삭제 $O(1)$, 노드 단위 연결 구조
앞/뒤 모두 삽입이 잦은 큐나 슬라이딩 윈도우	<code>deque</code>	<code>push_front</code> , <code>push_back</code> 모두 $O(1)$
요소 정렬이 자동으로 되어야 하고, 검색이 중요	<code>set</code> , <code>map</code>	이진 탐색 트리 기반 ($O(\log n)$)
키-값 저장이 필요하지만 정렬은 불필요	<code>unordered_map</code> , <code>unordered_set</code>	해시 기반, 평균 $O(1)$
후입선출 방식이 필요한 경우	<code>stack</code>	<code>deque</code> 기반 어댑터
선입선출 방식이 필요한 경우	<code>queue</code>	<code>deque</code> 기반 어댑터
우선순위대로 처리해야 하는 작업 큐	<code>priority_queue</code>	최대 힙 구조, 높은 우선순위가 먼저

31-10-2 메모리 관리 함수

- `reserve(n)`
 - `vector` 용량(capacity)을 미리 n 으로 확보하여 재할당 최소화
- `shrink_to_fit()`
 - 불필요한 capacity를 줄여 실제 크기에 맞춤 → 메모리 절약

31-10-3 실습: 대량 데이터 삽입 및 최적화

```
#include <iostream>
#include <vector>
#include <chrono>

int main() {
    std::vector<int> data;
    data.reserve(1000000); // 미리 메모리 확보

    auto start = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < 1000000; ++i) {
        data.push_back(i);
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::cout << "삽입 시간: "
              << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count()
              << "ms\n";

    data.shrink_to_fit();
    std::cout << "capacity 줄임 후 size: " << data.size() << ", capacity: " << data.capacity() << "\n";
    return 0;
}
```

31-10-4 실습 과제

1. `vector` 와 `list` 에 동일한 데이터 10만 개 삽입 시 시간 비교 실험
2. `deque` 로 앞/뒤 삽입 시 성능 측정
3. `shrink_to_fit()` 의 효과를 시각적으로 확인해보기 (용량 출력)

31-10-5 참고 자료

- [cppreference - vector::reserve](#)

- [cppreference - vector::shrink_to_fit](#)
- [STL Performance Guide \(Herb Sutter\)](#)

unordered_map 을 이용한 단어 빈도수 분석 프로그램 과제 설명

사용자로부터 한 줄의 문장을 입력받아, 문장에 포함된 각 단어가 몇 번씩 등장했는지를 계산하는 프로그램을 작성하세요. 단어별 등장 횟수는 `unordered_map` 을 활용하여 저장하고, 결과를 출력 형식에 맞춰 보여주세요.

- **입력:** 표준 입력을 통해 한 줄의 영어 문장 입력 (예: `"the quick brown fox jumps over the lazy dog"`)
- **출력:** 각 단어와 등장 횟수를 출력, 단어 순서는 상관 없음
- **대소문자 구분 없음** (`The` 와 `the` 는 같은 단어로 취급)
- **구두점(쉼표, 마침표 등)은 제외하고 분석**

```
// 표준 입력을 받아서 (영어문장) -this is a apple.
// 단어 별 등장 횟수를 출력하는 프로그램.
```

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <sstream> //string - stream
#include <string>
#include <unordered_map>
using namespace std;

string clean_word(const string &word)
{
    string cleaned;
    for (char ch : word)
    {
        if (isalpha(ch) || isdigit(ch))
            cleaned += tolower(ch);
    }
    return cleaned;
}

int main()
```

```

{
    string line;
    getline(cin, line);

    unordered_map<string, int> word_count;
    istringstream iss(line);
    string word;

    while (iss >> word)
    {
        string cleaned = clean_word(word);
        if (!cleaned.empty())
        {
            word_count[cleaned]++; // word_count[key] = value;
        }
    }
    for (const auto &[key, value] : word_count)
        cout << key << " : " << value << endl;
}

```

31-11 STL 알고리즘 개요 및 실습

31-11-1 STL 알고리즘 개요

- **STL 알고리즘**은 `<algorithm>` 헤더에 포함된 범용 함수들로, 컨테이너의 요소들을 다양한 방식으로 처리할 수 있도록 도와줍니다.
- 반복자(iterator)를 기반으로 작동하며, 대부분 ****범위 기반(first ~ last)****으로 동작합니다.
- 크게 **비변경 알고리즘**, **변경 알고리즘**, **정렬 및 수치 연산 알고리즘** 등으로 나뉩니다.

31-11-2 비변경 알고리즘

비변경 알고리즘은 컨테이너의 데이터를 **읽기만** 하며, 원본을 수정하지 않습니다.

- `std::find(begin, end, value)`
 - 특정 값을 찾아 반복자를 반환 (없으면 `end()` 반환)

- `std::count(begin, end, value)`
 - 값이 몇 번 나타나는지 개수 반환
- `std::all_of(begin, end, predicate)`
 - 모든 요소가 조건을 만족하는지 확인

예제

```
std::vector<int> v = {1, 2, 3, 4, 5};
bool all_positive = std::all_of(v.begin(), v.end(), [](int x) { return x > 0; });
```

31-11-3 변경 알고리즘

변경 알고리즘은 데이터를 복사하거나, 수정합니다.

- `std::copy(begin, end, dest_begin)`
 - 한 컨테이너의 요소를 다른 곳으로 복사
- `std::transform(begin, end, dest_begin, op)`
 - 변환 연산을 적용하며 복사
- `std::replace(begin, end, old_value, new_value)`
 - 지정된 값을 새 값으로 치환

예제

```
std::vector<int> v = {1, 2, 3, 4, 5};
std::vector<int> result(v.size());
std::transform(v.begin(), v.end(), result.begin(), [](int x) { return x * 10; });
```

31-11-4 실습: `find_if` 와 `transform` 활용

과제 목표:

- 짝수만 추출하여 새로운 벡터에 저장
- 각 값을 곱하여 또 다른 벡터에 저장

코드 예제

```
#include <iostream>
#include <vector>
```

```
#include <algorithm>

int main() {
    std::vector<int> nums = {1, 2, 3, 4, 5, 6};

    // 짝수만 필터링
    std::vector<int> evens;
    std::copy_if(nums.begin(), nums.end(), std::back_inserter(evens), [](int x)
    {
        return x % 2 == 0;
    });

    // 제곱하여 저장
    std::vector<int> squares(evens.size());
    std::transform(evens.begin(), evens.end(), squares.begin(), [](int x) {
        return x * x;
    });

    // 출력
    std::cout << "짝수 제곱 결과: ";
    for (int n : squares) std::cout << n << " ";
    std::cout << std::endl;
    return 0;
}
```

출력 결과

```
짝수 제곱 결과: 4 16 36
```

31-11-5 실습 과제

1. `std::count_if` 를 사용해 10보다 큰 수의 개수 구하기
2. `std::replace_if` 로 홀수를 0으로 바꾸기
3. `std::transform` 으로 문자열 벡터를 전부 대문자로 바꾸기 (힌트: `std::toupper`)

31-11-6 참고 자료

- [cppreference - algorithm](#)

- cplusplus.com - STL algorithms

31-12 STL 정렬 알고리즘 및 실습

31-12-1 정렬 알고리즘 개요

STL은 `<algorithm>` 헤더를 통해 다양한 정렬 알고리즘을 제공합니다. 이들은 **임의의 접근 반복자(Random Access Iterator)**를 요구하며 매우 효율적으로 동작합니다.

31-12-2 주요 정렬 알고리즘

1. `std::sort(begin, end)`
 - 기본 퀵소트 기반 알고리즘 (평균 $O(n \log n)$)
 - 기본 정렬 기준은 오름차순 (`operator<` 사용)
1. `std::stable_sort(begin, end)`
 - **안정 정렬(stable)**: 동일한 값일 경우 원래 순서를 유지
 - 병합 정렬 기반, 메모리 소모 큼 (최악 $O(n \log n)$)
1. `std::partial_sort(begin, middle, end)`
 - `begin` 부터 `middle` 까지만 정렬
 - 예: 상위 k개 요소만 정렬할 때

예제

```
std::vector<int> v = {5, 2, 9, 1, 5, 6};
std::sort(v.begin(), v.end());
```

31-12-3 사용자 정의 비교자

1. 기본 비교 함수 객체
 - `std::less<T>()` : 오름차순 (기본)
 - `std::greater<T>()` : 내림차순
1. 람다 표현식 또는 함수 객체

```
std::sort(v.begin(), v.end(), [](int a, int b) {
    return a > b; // 내림차순
});
```

31-12-4 실습: 구조체 벡터 정렬

구조체 정의

```
struct Student {
    std::string name;
    int score;
};
```

정렬 코드 예제

```
#include <iostream>
#include <vector>
#include <algorithm>

struct Student {
    std::string name;
    int score;
};

int main() {
    std::vector<Student> students = {
        {"Alice", 85},
        {"Bob", 92},
        {"Charlie", 78},
        {"David", 92}
    };

    // 점수 내림차순, 이름 오름차순으로 안정 정렬
    std::stable_sort(students.begin(), students.end(), [](const Student& a, const Student& b) {
        if (a.score == b.score)
            return a.name < b.name; // 이름 오름차순
    });
```

```

        return a.score > b.score; // 점수 내림차순
    });

    for (const auto& s : students) {
        std::cout << s.name << ": " << s.score << std::endl;
    }
    return 0;
}

```

출력 예시

```

Bob: 92
David: 92
Alice: 85
Charlie: 78

```

31-12-5 실습 과제

1. `partial_sort` 를 이용해 상위 3명의 점수만 정렬해 출력하기
2. `std::greater` 를 사용해 정수 벡터를 내림차순 정렬하기
3. 이름 기준 오름차순, 점수 기준 내림차순 정렬 (사용자 정의 비교자 연습)

31-12-6 참고 자료

- [cppreference - sort](#)
- [cppreference - stable_sort](#)
- [cppreference - partial_sort](#)

31-13 함수 객체와 std::function

31-13-1 함수 객체(Function Object, Functor)란?

- 정의: 함수처럼 동작할 수 있도록 `operator()` 를 오버로드한 객체
- 목적: 상태(state)를 가질 수 있는 함수 표현, 커스텀 동작 전달 등

예제:

```
struct Adder {
    int operator()(int a, int b) const {
        return a + b;
    }
};

Adder add;
int result = add(3, 5); // result = 8
```

- **장점:**
 - 상태 저장 가능 (**멤버 변수** 보관 가능)
 - 템플릿과 함께 사용 시 인라인 최적화 유리

31-13-2 표준 함수 래퍼 **std::function**

- **정의:** 다양한 호출 가능한 객체를 저장하고 호출할 수 있는 일반적인 함수 래퍼
- **<functional>** 헤더에 정의됨

사용 예:

```
#include <functional>

std::function<int(int, int)> func;

func = [](int a, int b) { return a + b; };
std::cout << func(2, 3); // 5
```

- **장점:**
 - 함수, 함수 포인터, 람다, 함수 객체 등 다양한 타입 저장 가능
 - 콜백 함수 형태로 유용하게 사용됨

31-13-3 콜백 구현과 활용 예

이벤트가 발생했을 때 미리 등록된 동작을 실행시키는 형태로 자주 사용

```

#include <iostream>
#include <functional>

void trigger_event(const std::function<void(int)>& handler) {
    std::cout << "이벤트 발생!\n";
    handler(42);
}

int main() {
    auto my_callback = [](int data) {
        std::cout << "콜백 실행됨. 데이터: " << data << std::endl;
    };

    trigger_event(my_callback);
    return 0;
}

```

31-13-4 실습: 이벤트 핸들러 시스템 구현

- 목표: 사용자로부터 이벤트 콜백을 등록받고, 이벤트 발생 시 호출되도록 구현

```

#include <iostream>
#include <functional>
#include <vector>

class EventManager {
public:
    void register_handler(std::function<void()> handler) {
        handlers.push_back(handler);
    }

    void fire_event() {
        std::cout << "[이벤트 발생]\n";
        for (auto& h : handlers) {
            h();
        }
    }
}

```

```
private:
    std::vector<std::function<void()>> handlers;
};

int main() {
    EventManager manager;

    manager.register_handler([](){ std::cout << "첫 번째 동작 실행\n"; });
    manager.register_handler([](){ std::cout << "두 번째 동작 실행\n"; });

    manager.fire_event();
    return 0;
}
```

출력 예시:

```
[이벤트 발생]
첫 번째 동작 실행
두 번째 동작 실행
```

31-13-5 실습 과제

1. `std::function<int(int)>` 을 이용해 여러 수학 함수(제곱, 두 배, 세 배 등)를 콜백으로 등록하고 실행해보세요.
2. 이벤트 매니저를 확장하여, 이벤트 이름별로 다른 콜백을 호출하도록 구현해보세요.
3. 멤버 함수 포인터를 `std::function` 에 래핑하여 객체의 메서드를 콜백으로 등록해보세요.

31-13-6 참고 자료

- [cppreference - std::function](#)
- [Modern C++ Callback Examples \(Herb Sutter\)](#)

31-14 고급 알고리즘과 병렬 실행

31-14-1 고급 STL 알고리즘 소개

C++ 표준 라이브러리는 기본 정렬 및 탐색 외에도 수치 연산과 조건 분할, 병합과 같은 고급 알고리즘을 제공합니다. 이들은 `<numeric>`, `<algorithm>` 헤더에 포함되어 있습니다.

31-14-2 주요 알고리즘 설명

1. `std::accumulate`

- 누적 합 또는 일반적인 축적 작업 수행
- `<numeric>` 헤더에 존재

```
#include <numeric>
int sum = std::accumulate(vec.begin(), vec.end(), 0);
```

1. `std::partition`

- 조건에 따라 컨테이너를 두 부분으로 분할
- 반환값은 경계 반복자

```
auto it = std::partition(vec.begin(), vec.end(), [](int x){ return x % 2 == 0; });
```

1. `std::merge`

- 정렬된 두 범위를 하나의 정렬된 범위로 병합
- 두 입력 범위는 반드시 정렬되어 있어야 함

```
std::merge(a.begin(), a.end(), b.begin(), b.end(), std::back_inserter(result));
```

31-14-3 병렬 알고리즘 (C++17~)

- `<execution>` 헤더를 통해 병렬화 가능
- `std::execution::par`, `par_unseq` 등 정책(policy) 지정
- 동일한 알고리즘에 대해 병렬 처리를 지시

예제:

```
#include <execution>
```

```
std::sort(std::execution::par, vec.begin(), vec.end());
```

병렬 알고리즘은 하드웨어 환경, 스레드 수, 데이터 크기에 따라 성능 향상이 다를 수 있음

31-14-4 실습: **accumulate** 를 활용한 통계 계산

```
#include <iostream>
#include <vector>
#include <numeric>
#include <cmath>

int main() {
    std::vector<int> data = {1, 2, 3, 4, 5};

    double mean = std::accumulate(data.begin(), data.end(), 0.0) / data.size();

    double sq_sum = std::accumulate(data.begin(), data.end(), 0.0,
    [mean](double acc, int val) {
        return acc + (val - mean) * (val - mean);
    });

    double variance = sq_sum / data.size();

    std::cout << "평균: " << mean << "\n";
    std::cout << "분산: " << variance << "\n";
    return 0;
}
```

31-14-5 실습: 병렬 알고리즘 테스트

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <execution>
```

```

#include <random>
#include <chrono>

int main() {
    std::vector<int> vec(1'000'000);
    std::iota(vec.begin(), vec.end(), 0);
    std::shuffle(vec.begin(), vec.end(), std::mt19937{std::random_device{}}());

    auto start = std::chrono::high_resolution_clock::now();
    std::sort(std::execution::par, vec.begin(), vec.end());
    auto end = std::chrono::high_resolution_clock::now();

    std::cout << "병렬 정렬 소요 시간: "
              << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count()
              << "ms\n";
    return 0;
}

```

병렬 실행은 지원되지 않는 컴파일러(C++17 미만 또는 MinGW 등)에서는 오류가 발생할 수 있으니 주의 필요

31-14-6 실습 과제

1. `std::partition` 을 이용해 짝수/홀수 구분된 벡터를 만들고 출력해보기
2. `std::merge` 로 두 정렬된 벡터를 병합하는 함수 작성하기
3. `std::reduce` 를 사용한 병렬 합산 구현 (C++17 이상)

31-14-7 참고 자료

- [cppreference - numeric algorithms](#)
- [cppreference - execution policies](#)
- [ISO C++ std::execution 소개](#)

숙제: `std::sort` 와 사용자 정의 비교자를 활용한 데이터 정렬 프로그램

과제 설명

학생들의 이름과 점수를 저장한 구조체 벡터를 정렬하는 프로그램을 작성하세요.

기본 정렬 기준은 점수 내림차순이며, 점수가 같으면 이름 오름차순으로 정렬해야 합니다.

- `std::sort` 또는 `std::stable_sort` 사용
- 람다식 또는 함수 객체를 이용한 사용자 정의 비교자 사용

체크포인트

항목	설명
구조체 정의	이름(name)과 점수(score)를 가진 <code>Student</code> 구조체 작성
사용자 정의 비교자 구현 여부	람다 또는 함수 객체를 통해 다중 기준 정렬 구현
<code>std::sort</code> 또는 <code>stable_sort</code> 사용	<code><algorithm></code> 의 정렬 알고리즘 사용
정렬 결과 출력	정렬된 순서대로 이름과 점수 출력 (<code>이름: 점수</code> 형식)
안정성 고려 여부	<code>stable_sort</code> 사용 시 이름 순서가 유지되는지 확인 가능 (선택 과제)

예시 입력

```
cpp
복사편집
std::vector<Student> students = {
    {"Alice", 90},
    {"Bob", 95},
    {"Charlie", 90},
    {"David", 80}
};
```

예시 출력

```
makefile
복사편집
```

Bob: 95
Alice: 90
Charlie: 90
David: 80

✅ 정답 예시 코드

```
cpp
복사편집
#include <iostream>
#include <vector>
#include <algorithm>

struct Student {
    std::string name;
    int score;
};

int main() {
    std::vector<Student> students = {
        {"Alice", 90},
        {"Bob", 95},
        {"Charlie", 90},
        {"David", 80}
    };

    std::sort(students.begin(), students.end(), [](const Student& a, const Student& b) {
        if (a.score == b.score)
            return a.name < b.name; // 이름 오름차순
        return a.score > b.score; // 점수 내림차순
    });

    for (const auto& s : students) {
        std::cout << s.name << ": " << s.score << std::endl;
    }
}
```



```
return 0;  
}
```

🎯 심화 과제 (선택)

- `std::stable_sort` 를 사용하여 이름 정렬의 **안정성 유지**를 실험해보세요.
- 이름 기준 오름차순, 점수 기준 오름차순으로 정렬하는 버전도 만들어보세요.
- 파일로부터 학생 정보를 입력받아 정렬하는 기능을 구현해보세요.